

NUNNUN API SPEC

좋아. 기존 `API_SPEC` 의 구조는 유지하면서, 방금 확정된 **취침 유도 알림 정책**까지 포함한 최신 노션 북붙용 API 명세로 다시 정리할게.

기존 명세에는 데모 로그인, 요일별 기상목표, 수동 DND, 단일 깨우기 그룹, 인증 2회, `NEEDS_HELP`, 성공 후 30분 쿨다운, 사진 8시간 유지, 셀프 인증까지 정의되어 있었고, `POST /me/sleep` 은 존재하지만 알림과의 연결 규칙은 아직 없었다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF NUNNUN_API_SPEC_v1_REVISIED.pdfPDF 이번 버전에서는 기상목표시간 기반 취침 유도 알림 + 알림의 **잠자기** 버튼 + 해당 사이클 알림 중단을 추가한다.

NUNNUN API SPEC

0. 현재 개발 범위

개발함

- 데모 로그인
- FCM 디바이스 등록
- 요일별 기상목표시간
- 기상목표 기반 취침 유도 알림
- 알림의 **잠자기** 버튼
- 수동 DND
- 단일 깨우기 그룹
- 깨우기 그룹 정원 4명
- 그룹 상세
- 그룹원 깨우기
- 오늘의 포즈
- 사진 인증
- AI 포즈 판정
- 인증 총 2회
- `NEEDS_HELP`
- 성공 후 30분 쿨다운
- 성공 인증사진 8시간 유지
- `AWAKE / SLEEPING` 계산
- 셀프 인증

이번 개발에서 제외

- 여러 깨우기 그룹

- 그룹 탈퇴
- AI 친구
- 실제 결제
- 4명 → 8명 확장 기능
- 그룹 간 인증 공유
- Roommate 신규 개발
- Reward

1. Endpoint Summary

Domain	Method	URL	기능	우선순위
Demo Auth	GET	/demo-accounts	데모 계정 목록	P0
Demo Auth	POST	/auth/demo-login	데모 계정 로그인	P0
Auth	POST	/auth/reissue	토큰 재발급	P0
Auth	POST	/auth/logout	로그아웃	P0
User	GET	/users/me	내 정보 조회	P1
User	PATCH	/users/me	내 정보 수정	P2
User	DELETE	/users/me	회원탈퇴	P2
Device	POST	/devices	FCM 토큰 등록	P0
My	GET	/me/today	오늘 화면 조회	P1
Schedule	POST	/me/fixed-schedules/analyze	시간표 이미지 AI 분석	P2
Schedule	POST	/me/fixed-schedules/import	OCR/캘린더 일정 저장	P2
Schedule	GET	/me/fixed-schedules	고정 시간표 조회	P2
Schedule	POST	/me/fixed-schedules	일정 추가	P2
Schedule	PATCH	/me/fixed-schedules/{id}	일정 수정	P2
Schedule	DELETE	/me/fixed-schedules/{id}	일정 삭제	P2
My	PATCH	/me/today/bed-time	오늘 취침시간 변경	P2
My	PATCH	/me/today/return-time	오늘 귀가시간 변경	P2
Wake Target	GET	/me/wake-targets	요일별 기상목표 조회	P0
Wake Target	POST	/me/wake-targets	요일별 기상목표 추가/변경	P0
Wake Target	DELETE	/me/wake-targets/{dayOfWeek}	요일 기상목표 해제	P1
Sleep	POST	/me/sleep	잠들기 기록 + 현재 취침 알림 중단	P0

Domain	Method	URL	기능	우선순위
DND	GET	/me/dnd-windows	방해금지 시간대 조회	P0
DND	POST	/me/dnd-windows	방해금지 시간대 추가	P0
DND	DELETE	/me/dnd-windows/{id}	방해금지 시간대 삭제	P1
Group	GET	/groups	내 깨우기 그룹 목록	P0
Wake Group	POST	/wake-groups	깨우기 그룹 생성	P0
Wake Group	GET	/wake-groups/{id}	깨우기 그룹 상세 조회	P0
Wake Group	PATCH	/wake-groups/{id}	그룹 이름 변경	P1
Wake Group	GET	/wake-groups/preview?code={code}	초대코드 검증/미리보기	P0
Wake Group	POST	/wake-groups/join	깨우기 그룹 참여	P0
Wake Group	GET	/wake-groups/{id}/invite-code	초대코드 조회	P1
Wake	POST	/wake-groups/{id}/members/{userId}/wake	그룹원 깨우기	P0
Wake	GET	/wake-requests/{id}	깨우기 요청/오늘 포즈 조회	P0
Wake	POST	/wake-requests/{id}/proof	인증사진 업로드 및 AI 판정	P0
Wake	POST	/me/self-verify	셀프 인증 시작	P0
Stats	GET	/me/stats	기상 통계 조회	P2
Sleep	POST	/me/sleep-feedback	수면 만족도 등록	P2

기존 명세의 주요 Wake Endpoint 구조는 그대로 유지한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF
NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

2. 핵심 정책 요약

기상목표시간

사용자가 마이페이지에서 **요일별로 직접 설정**한다.

예:

월요일, 07:30
화요일, 08:00
수요일, 09:00

해당 요일 값을 수정하면 기존 값이 완전히 변경되고 앞으로 매주 반복 적용된다.

취침 유도 알림

다음으로 적용될 기상목표시간을 **T** 라고 한다.

첫 알림 = T - 9시간
알림 간격 = 90분
마지막 알림 = T - 1시간 30분

알림은 매일 자동 반복한다.

첫 번째 알림:

취침 1시간 전이에요.

그 이후:

기상까지 N시간 N분 남았어요.

마지막 알림은:

기상까지 1시간 30분 남았어요.

이다.

마지막 알림 이후에는 해당 기상목표시간에 대해 더 이상 취침 유도 알림을 보내지 않는다.

잠자기 버튼

모든 취침 유도 알림에는:

[잠자기]

버튼이 존재한다.

사용자가 누르면:

POST /me/sleep

을 호출한다.

그 결과:

수면 시작 기록
+
현재 취침 사이클의 남은 알림 취소

가 이루어진다.

하지만 알림 자체를 영구적으로 끄는 것은 아니다.

다음 기상목표시간의 **9시간 전부터 다시 알림이 시작된다.**

깨우기

기상목표시간과 관계없이 깨우기 가능.

차단 조건은 두 가지뿐이다.

1. 상대방 DND
2. 상대방 성공 인증 후 30분 쿨다운

기존 **SENT** 요청이 있어도 새로운 깨우기 요청 생성 가능.

인증

- 1회차 = 최초 인증
- 2회차 = 재시도

총 2회.

- 70점 이상 = SUCCESS
- 70점 미만 = FAIL

두 번째 실패:

NEEDS_HELP

성공:

VERIFIED
30분 쿨다운
사진 8시간 유지

3. Demo Auth

GET **/demo-accounts**

데모 계정 목록 조회.

Response 200

```
{
  "accounts": [
    {
      "id": 1,
      "nickname": "눈눈",
      "avatar_url": "https://..."
    },
  ],
}
```

```
{
  "id": 2,
  "nickname": "지우",
  "avatar_url": "https://..."
}
```

POST `/auth/demo-login`

Request

```
{
  "demo_account_id": 1
}
```

Response 200

```
{
  "access_token": "...",
  "refresh_token": "...",
  "user": {
    "id": 1,
    "nickname": "눈눈",
    "avatar_url": "https://..."
  }
}
```

`/auth/reissue`, `/auth/logout` 은 기존 JWT 흐름을 그대로 사용한다.
NUNNUN_API_SPEC_v1_REVISED.pdfPDF

4. Device / FCM

POST `/devices`

사용자의 FCM Token을 서버에 등록한다.

취침 유도 알림과 깨우기 알림 발송에 사용한다.

Request 예시

```
{
  "fcm_token": "firebase-device-token",
  "platform": "ANDROID"
}
```

Response 200

```
{
  "registered": true
}
```

```
}
```

동일 토큰이 다시 등록되면 새 row를 무조건 생성하기보다는 기존 정보를 갱신하는 방향으로 처리한다.

5. 요일별 기상목표시간

기존 명세에서도 일별 직접 수정 방식 대신 `GET/POST /me/wake-targets` 로 요일별 반복 목표를 사용하도록 변경되어 있다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

GET `/me/wake-targets`

Response 200

```
{
  "targets": [
    {
      "day_of_week": "MONDAY",
      "display_day": "월요일",
      "target_wake_time": "07:30"
    },
    {
      "day_of_week": "TUESDAY",
      "display_day": "화요일",
      "target_wake_time": "08:00"
    },
    {
      "day_of_week": "WEDNESDAY",
      "display_day": "수요일",
      "target_wake_time": null
    }
  ]
}
```

POST `/me/wake-targets`

요일별 기상목표 추가 또는 변경.

Request

```
{
  "text": "월요일, 07:30"
}
```

Response 200

```
{
  "day_of_week": "MONDAY",
  "target_wake_time": "07:30",
  "display_text": "월요일, 07:30"
}
```

입력 형식

요일, HH:mm

올바른 예:

월요일, 07:30
금요일, 09:00

잘못된 예:

월요일 07:30
월, 07:30
월요일, 7:30
월요일, 오전 7시

오류

INVALID_WAKE_TARGET_FORMAT

메시지:

예시와 같은 형식으로 다시 입력해주세요.
예) 월요일, 07:30

기상목표시간 변경 시 알림

요일별 기상목표를 변경하면 아직 발송되지 않은 해당 목표의 취침 유도 알림 일정도 새로운 목표시간에 맞춰 다시 계산한다.

예:

기존 화요일 목표 = 09:00

변경
→ 화요일 목표 = 08:00

아직 도래하지 않은 다음 화요일의 알림도:

09:00 기준 일정 X
08:00 기준 일정 0

으로 갱신한다.

DELETE `/me/wake-targets/{dayOfWeek}`

해당 요일 목표 삭제.

예:

```
DELETE /me/wake-targets/MONDAY
```

목표를 삭제하면 해당 목표시간에 연결되어 아직 발송되지 않은 취침 유도 알람도 취소한다.

6. 다음 기상목표시간 결정 규칙

현재 시각 이후 가장 먼저 도래하는 **설정된 요일별 기상목표시간**을 사용한다.

예:

```
일요일 목표 = 09:00  
월요일 목표 = 07:30
```

현재:

```
일요일 08:00
```

이면:

```
next_target_at = 오늘 09:00
```

현재:

```
일요일 10:00
```

이면:

```
next_target_at = 월요일 07:30
```

이다.

목표가 없는 요일

해당 요일은 건너뛴다.

예:

```
월요일 = 미설정  
화요일 = 08:00
```

이면 월요일에는 별도의 월요일 목표가 없고 다음 목표는:

화요일 08:00

이다.

단, 그룹 카드에서 **오늘 요일의 목표시간이 없는 경우에는** 기존 정책대로:

목표까지 남은 시간 계산 X
목표 +15분 NEEDS_HELP 계산 X
그 요일 목표 기반 취침 전환 계산 X

으로 처리한다.

다른 사람의 깨우기 및 셀프 인종은 그대로 가능하다.

7. 취침 유도 알림

기본 규칙

기상목표시간을 **T** 라고 한다.

수면시간을 8시간으로 가정한다.

취침 기준시간:

T - 8시간

첫 취침 유도 알림은 취침 기준시간 1시간 전이므로:

T - 9시간

에 발송한다.

이후:

90분

간격으로 발송한다.

마지막 알림:

T - 1시간 30분

이후 해당 기상목표에 대해서는 알림을 보내지 않는다.

8. 알림 예시 — 목표 09:00

기상목표:

09:00

취침 기준:

01:00

첫 알림:

00:00

알림 일정:

시각	기상까지	내용
00:00	9시간	취침 1시간 전이에요.
01:30	7시간 30분	기상까지 7시간 30분 남았어요.
03:00	6시간	기상까지 6시간 남았어요.
04:30	4시간 30분	기상까지 4시간 30분 남았어요.
06:00	3시간	기상까지 3시간 남았어요.
07:30	1시간 30분	기상까지 1시간 30분 남았어요.

09:00

에는 별도의 취침 유도 알림을 보내지 않는다.

9. 알림 예시 — 목표 07:30

기상목표:

07:30

첫 알림:

전날 22:30

알림 일정:

시각	내용
전날 22:30	취침 1시간 전이에요.
00:00	기상까지 7시간 30분 남았어요.
01:30	기상까지 6시간 남았어요.
03:00	기상까지 4시간 30분 남았어요.

시각	내용
04:30	기상까지 3시간 남았어요.
06:00	기상까지 1시간 30분 남았어요.

날짜가 바뀌더라도 동일하게 계산한다.

10. 취침 유도 알림 특성

취침 유도 알림은 소리와 진동이 없는 알림이다.

```
Sound = OFF
Vibration = OFF
```

깨우기 알림과 같은 Notification Channel을 사용하지 않는 것이 좋다.

예:

```
BEDTIME_REMINDER
```

전용 Android Notification Channel을 사용하고 해당 Channel의:

```
sound = none
vibration = false
```

로 설정한다.

무음/무진동 자체는 Android 앱의 Notification Channel 설정도 필요하다. 백엔드는 알림 종류를 `BEDTIME_REMINDER` 로 구분해서 내려준다.

11. FCM 취침 알림 Payload

예시:

```
{
  "type": "BEDTIME_REMINDER",
  "title": "눈눈",
  "body": "기상까지 6시간 남았어요.",
  "silent": true,
  "action": "SLEEP",
  "target_wake_at": "2026-08-17T09:00:00+09:00"
}
```

첫 알림:

```
{
  "type": "BEDTIME_REMINDER",
  "title": "눈눈",
  "body": "취침 1시간 전이에요.",
  "silent": true,
```

```
"action": "SLEEP",
"target_wake_at": "2026-08-17T09:00:00+09:00"
}
```

프론트는 `action = SLEEP` 을 보고 알림 하단에:

```
[ 잠자기 ]
```

버튼을 표시한다.

12. 알림 `잠자기` 버튼

알림에서:

```
[ 잠자기 ]
```

버튼을 누르면:

```
POST /me/sleep
```

을 호출한다.

별도의 알림 전용 잠자기 API를 만들지 않고 기존 Sleep API를 재사용한다.

13. POST `/me/sleep`

사용자가 앱 또는 알림창에서 잠자기를 선택했을 때 호출한다.

Request

앱 내부 잠들기 버튼:

```
{
  "source": "APP"
}
```

알림창의 잠자기 버튼:

```
{
  "source": "NOTIFICATION"
}
```

`source` 가 굳이 필요하지 않은 구현이라면 생략할 수 있지만, 로그 및 디버깅을 위해 구분하는 것을 권장한다.

Response 201

```
{
  "sleep_session_id": 301,
  "started_at": "2026-08-17T03:00:00+09:00",
  "bedtime_reminders_cancelled": true
}
```

14. 잠자기 호출 시 서버 처리

`POST /me/sleep` 이 성공하면 서버는:

1. sleep_sessions 생성
2. started_at = 현재 시각 기록
3. 현재 취침 사이클의 PENDING BEDTIME_REMINDER 취소
4. 다음 취침 사이클은 건드리지 않음

으로 처리한다.

예:

```
기상목표 = 09:00

00:00 알림
01:30 알림
03:00 알림
→ 03:00에서 잠자기 클릭
```

그러면:

```
04:30 취소
06:00 취소
07:30 취소
```

된다.

15. 취침 사이클의 의미

한 취침 알림 사이클은 하나의 특정 기상목표 시각을 기준으로 한다.

예:

```
target_wake_at
= 2026-08-17 09:00
```

에 대한 알림들은 모두 같은 취침 사이클이다.

```
00:00
01:30
03:00
04:30
```

06:00
07:30

이 모두:

2026-08-17 09:00 기상

을 위한 알림이다.

사용자가 그중 하나에서 **잠자기**를 누르면 이 사이클에 속한 아직 발송되지 않은 알림만 취소한다.

16. 다음날 다시 알림 시작

잠자기는 알림을 영구 OFF 하는 기능이 아니다.

예:

월요일 목표 = 09:00
화요일 목표 = 09:00

월요일 기상 사이클에서 잠자기를 눌러 남은 알림을 취소했더라도:

화요일 09:00 - 9시간
= 화요일 00:00

부터 새로운 알림 사이클이 시작된다.

즉:

오늘의 남은 알림만 중단

이다.

17. 기상목표시간이 없는 경우의 알림

다음으로 적용할 기상목표시간이 없다면 취침 유도 알림도 생성하지 않는다.

기상목표 없음
→ BEDTIME_REMINDER 없음

사용자가 나중에 기상목표를 등록하면 새로운 목표를 기준으로 알림 일정을 계산한다.

18. DND

DND 정책은 기존과 동일하다. 기존 명세에서도 사용자가 직접 등록한 DND만 사용하고, 고정 시간표와 자동 연결하지 않도록 정의되어 있다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

GET /me/dnd-windows

Response 200

```
{
  "windows": [
    {
      "id": 10,
      "day_of_week": "MONDAY",
      "start_time": "08:00",
      "end_time": "11:00",
      "display_text": "월요일, 08:00~11:00"
    }
  ]
}
```

POST /me/dnd-windows

Request

```
{
  "text": "월요일, 08:00~11:00"
}
```

형식:

요일, HH:mm~HH:mm

이번 MVP:

start_time < end_time

만 허용.

DELETE /me/dnd-windows/{id}

등록 DND 삭제.

DND는 다른 사람의 깨우기만 막는다.

다음에는 영향을 주지 않는다.

셀프 인증
취침 유도 알림

즉 취침 알림은 DND 여부와 관계없이 발송한다.

19. Wake Group

POST `/wake-groups`

Request

```
{
  "name": "아침 야호"
}
```

Response 201

```
{
  "id": 10,
  "name": "아침 야호",
  "invite_code": "F2K8G3",
  "capacity": 4,
  "current_members": 1
}
```

한 사용자는 깨우기 그룹 최대 1개.

이미 그룹이 있으면:

```
ACTIVE_WAKE_GROUP_EXISTS
```

GET `/wake-groups/preview?code={code}`

Response 200

```
{
  "valid": true,
  "reason": null,
  "group_name": "아침 야호",
  "current_members": 2,
  "capacity": 4
}
```

실패 Reason:

```
INVALID_CODE
GROUP_FULL
ALREADY_IN_WAKE_GROUP
ALREADY_MEMBER
```

POST `/wake-groups/join`

Request

```
{
  "invite_code": "F2K8G3"
}
```

현재:

```
capacity = 4
```

결제 및 8명 확장은 이번 범위 제외.

PATCH `/wake-groups/{id}`

Request

```
{
  "name": "일어나자"
}
```

현재 그룹 구성원이면 이름 변경 가능.

그룹장 권한은 별도로 사용하지 않는다.

기존 Wake Group API 정책을 유지한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

20. Wake Group Detail

GET `/wake-groups/{id}`

Response 200

```
{
  "id": 10,
  "name": "아침 야호",
  "invite_code": "F2K8G3",
  "capacity": 4,
  "current_members": 2,
  "members": [
    {
      "user_id": 1,
      "nickname": "눈눈",
      "avatar_url": "https://...",
      "is_me": true,

      "target_wake_time": "09:00",
      "next_target_at": "2026-08-17T09:00:00+09:00",

      "remaining_to_target": {
        "value": 3,
        "unit": "HOUR"
      },

      "state": "AWAKE",
    }
  ]
}
```

```

    "actual_wake_time": "09:03",

    "proof_image_url": "https://...",
    "proof_expires_at": "2026-08-16T17:03:00+09:00",

    "can_wake": false,
    "block_reason": "COOLDOWN",
    "wake_available_at": "2026-08-16T09:33:00+09:00"
  }
]
}

```

state:

```

NORMAL
AWAKE
SLEEPING
NEEDS_HELP

```

기존 상세 응답과 남은 시간 계산 규칙은 그대로 사용한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

21. 목표까지 남은 시간

60분 이상:

시간 단위 올림

예:

```

181분 → 4시간
121분 → 3시간
120분 → 2시간
61분 → 2시간
60분 → 1시간

```

60분 미만:

```

59분 → 59분
32분 → 32분
1분 → 1분

```

목표시간을 지났지만 목표 +15분 전:

```

0분
목표까지

```

22. 그룹원 깨우기

POST `/wake-groups/{id}/members/{userId}/wake`

Response 201

```
{
  "wake_request_id": 501,
  "status": "SENT",
  "requested_at": "2026-08-16T07:32:00+09:00"
}
```

차단 조건

DND
→ WAKE_BLOCKED_DND

또는:

최근 SUCCESS verified_at + 30분 이내
→ WAKE_COOLDOWN

이 두 가지뿐이다.

기상목표시간은 깨우기 가능 여부를 제한하지 않는다.

기존 **SENT** 요청도 새로운 깨우기를 막지 않는다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

23. 오늘의 포즈

그룹별 하루 하나.

해당 그룹에서 그날 처음:

깨우기
또는
셀프 인증

이 발생했을 때 생성한다.

이미 오늘 포즈가 있으면 기존 포즈를 사용한다.

24. GET `/wake-requests/{id}`

Response 200

```
{
  "id": 501, "group_id": 7,
  "status": "SENT",

  "sender": {
```

```

    "id": 2,
    "nickname": "지우"
  },

  "receiver": {
    "id": 1,
    "nickname": "눈눈"
  },

  "requested_at": "2026-08-16T07:32:00+09:00",

  "pose": {
    "date": "2026-08-16",
    "description": "양손으로 머리 위 하트를 만들어주세요"
  },

  "attempts_used": 0,
  "remaining_attempts": 2
}

```

기존 Response 구조를 유지한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

25. 인증사진 + AI 판정

POST `/wake-requests/{id}/proof`

```

Content-Type: multipart/form-data
field = image

```

최대 2회 호출 가능.

SUCCESS

```

{
  "wake_request_id": 501,
  "attempt_no": 1,

  "pose_match_score": 82,
  "pose_match_result": "SUCCESS",

  "request_status": "VERIFIED",

  "can_retry": false,
  "remaining_attempts": 0,

  "verified_at": "2026-08-16T09:03:00+09:00",
  "cooldown_until": "2026-08-16T09:33:00+09:00",
  "proof_expires_at": "2026-08-16T17:03:00+09:00"
}

```

1회차 FAIL

```
{
  "wake_request_id": 501,
  "attempt_no": 1,

  "pose_match_score": 42,
  "pose_match_result": "FAIL",

  "request_status": "SENT",

  "can_retry": true,
  "remaining_attempts": 1
}
```

2회차 FAIL

```
{
  "wake_request_id": 501,
  "attempt_no": 2,

  "pose_match_score": 31,
  "pose_match_result": "FAIL",

  "request_status": "NEEDS_HELP",

  "can_retry": false,
  "remaining_attempts": 0
}
```

판정 기준:

```
score >= 70 → SUCCESS
score < 70 → FAIL
```

70은 서버 설정값.

기존 인증 성공/실패 정책은 유지한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

26. SUCCESS 정책

인증 성공:

```
status = VERIFIED
```

실제 기상시간:

```
verified_at
```

성공 쿨다운:

verified_at + 30분

사진:

verified_at + 8시간

후 삭제.

사진만 사라지고 성공 기록 자체는 유지한다.

27. NEEDS_HELP

Case 1

인증 2회 모두 FAIL
→ 즉시 NEEDS_HELP

Case 2

오늘 기상목표시간이 존재하고:

기상목표시간 + 15분

까지 SUCCESS가 없으면:

NEEDS_HELP

목표시간 없는 요일에는 Case 2를 적용하지 않는다.

기존 명세의 NEEDS_HELP 규칙과 동일하다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

28. AWAKE / SLEEPING

SUCCESS 이후:

AWAKE

다음 기상목표시간의:

8시간 전

부터:

SLEEPING

으로 계산한다.

예:

다음 목표 = 09:00

01:00부터 SLEEPING

취침 알림과의 관계

같은 09:00 목표라면:

00:00

첫 취침 유도 알림
"취침 1시간 전이에요."

01:00

SLEEPING 상태 기준시각

01:30

두 번째 취침 유도 알림
"기상까지 7시간 30분 남았어요."

즉:

목표 - 9시간

→ 첫 알림

목표 - 8시간

→ 취침 기준 / SLEEPING 기준

목표 - 1시간 30분

→ 마지막 알림

으로 서로 연결된다.

29. Self Verify

POST /me/self-verify

현재 사용자에게 가입된 하나의 깨우기 그룹을 사용한다.

Response 201

```
{
  "wake_request_id": 610,
  "status": "SENT",
  "self_verify": true,
```



```
"pose": {  
  "date": "2026-08-16",  
  "description": "한 손은 브이, 다른 손은 볼 옆에 대주세요"  
}
```

이후:

```
POST /wake-requests/{id}/proof
```

를 그대로 사용한다.

셀프 인증 성공도:

```
30분 쿨다운  
사진 8시간
```

적용.

DND는 셀프 인증을 차단하지 않는다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

30. GET `/me/today`

GET /me/today - 오늘 화면 조회 [P1]
기존 Today 데이터 구조를 유지합니다.
resolved_target_wake_time:
오늘 Asia/Seoul 기준 요일에 설정된 Weekly Wake Target입니다.
HH:mm 형식으로 반환하며, 오늘 설정이 없으면 null입니다.
Canonical Wake Target은 weekly_wake_targets입니다.
daily_routines.target_wake_time을 fallback으로 사용하지 않습니다.

next_target_at:
현재 시각보다 엄격하게 미래에 있는 가장 가까운 Weekly Wake Target의 시각입니다.
현재 시각과 기상목표 시각이 정확히 같으면
해당 목표는 이미 도달한 것으로 보고 다음 설정 목표를 찾습니다.
설정된 미래 목표가 없으면 null입니다.
예:
"next_target_at": "2026-08-18T07:30:00+09:00"

31. GET `/me/stats`

통계 계산 기준

통계 기간은 별도의 7일/30일 제한 없이 현재 DB에 보존된 전체 기록을 사용합니다.

모든 날짜 및 시간 계산의 기준 시간대는 Asia/Seoul입니다.

success_rate

현재 사용자가 receiver인 WakeRequest 중 인증 결과가 최종 확정된 요청의 성공 비율입니다.

계산 대상:

- VERIFIED
- NEEDS_HELP

제외:

- SENT

계산식:

$$\text{VERIFIED 개수} / (\text{VERIFIED 개수} + \text{NEEDS_HELP 개수}) \times 100$$

규칙:

- Self Verify(sender_id = receiver_id)도 동일하게 포함합니다.
- target +15분으로 조회 시 파생되는 NEEDS_HELP는 실제 WakeRequest가 없는 경우 통계에 포함하지 않습니다.
- 같은 WakeRequest의 인증 재시도는 하나의 WakeRequest로 계산합니다.
- 성공 사진이 8시간 후 삭제되어도 VERIFIED 기록은 유지되므로 성공으로 계산합니다.
- 계산 가능한 요청이 없으면 0.0입니다.
- 소수점 첫째 자리까지 반올림합니다.

avg_gap_minutes

성공 인증 시각과 해당 WakeRequest가 생성될 당시 적용된 기상목표 시각의 평균 차이입니다.

기상목표는 wake_requests.target_wake_at 스냅샷을 사용합니다.

각 기록의 계산식:

$$\text{gap_minutes} = \text{verified_at} - \text{target_wake_at}$$

예:

- target 07:30 / verified 07:20 → -10분
- target 07:30 / verified 07:30 → 0분
- target 07:30 / verified 07:45 → +15분

의미:

- 음수: 목표보다 일찍 인증
- 0: 목표시각에 인증
- 양수: 목표보다 늦게 인증

계산 대상:

- WakeRequest.status = VERIFIED

- WakeProof.pose_match_result = SUCCESS
- verified_at 존재
- target_wake_at 존재

제외:

- target_wake_at = NULL
- SENT
- NEEDS_HELP
- FAIL

현재 Weekly Wake Target을 과거 기록에 소급 적용하지 않습니다.

기존 데이터 중 target_wake_at이 없는 기록은 avg_gap_minutes에서 제외합니다.

계산 가능한 기록이 없으면 0.0입니다.

산술평균을 소수점 첫째 자리까지 반올림합니다.

streak_days

Asia/Seoul 기준으로 SUCCESS 인증이 존재하는 연속된 달력 날짜 수입니다.

성공일:

해당 날짜에 최소 하나의

VERIFIED + SUCCESS + verified_at 기록이 존재하는 날짜

규칙:

- 하루에 SUCCESS가 여러 개 있어도 1일로 계산합니다.
- Self Verify SUCCESS도 포함합니다.
- 성공 이미지가 삭제되어 image_object_key가 NULL이어도 성공일입니다.
- Weekly Wake Target 설정 여부는 streak 계산에 사용하지 않습니다.

오늘 SUCCESS가 있으면 오늘부터 과거 방향으로 계산합니다.

오늘 SUCCESS가 아직 없으면 오늘이 진행 중이라는 이유만으로
기존 streak를 끊지 않고 어제부터 계산합니다.

계산 중 SUCCESS가 없는 날짜를 만나면 streak가 종료됩니다.

성공 기록이 없으면 0입니다.

32. 취침 유도 알림 서버 흐름

```

요일별 기상목표 설정
↓
다음 target_wake_at 계산
↓
target_wake_at - 9시간부터 알림 일정 생성
↓
90분 간격
↓
target_wake_at - 1시간 30분까지 생성
↓
FCM 발송

```

↓
무음 / 무진동 알림 표시
↓
사용자가 [잠자기] 클릭?

NO

다음 예약 알림 계속 발송

YES

POST /me/sleep
↓
SleepSession 생성
↓
현재 target_wake_at에 속한
남은 PENDING BEDTIME_REMINDER 취소
↓
현재 취침 사이클 종료
↓
다음 기상목표 사이클은 정상 생성

33. 기상목표 변경과 알림 재계산

예:

현재
화요일 09:00

알림:

00:00
01:30
03:00
04:30
06:00
07:30

사용자가 화요일 목표를:

08:00

으로 변경하면 아직 발송되지 않은 기존 알림을 취소하고 새로운 목표 기준으로 재계산한다.

새 알림:

전날 23:00
00:30
02:00
03:30

05:00
06:30

단, 이미 과거가 된 시각의 알림을 뒤늦게 발송하지 않는다.

예를 들어 현재가:

02:30

이라면:

23:00 X
00:30 X
02:00 X

03:30부터 정상 발송

으로 처리한다.

34. 알림 중복 방지

같은 사용자, 같은 기상목표, 같은 예약시각에 대해 같은 `BEDTIME_REMINDER` 를 중복 생성하거나 발송하지 않는다.

개념적으로:

```
user
+
target_wake_at
+
scheduled_at
+
type = BEDTIME_REMINDER
```

조함은 하나만 존재하도록 관리한다.

이 제약은 이후 DB 설계에 반영한다.

35. 시간대

현재 서비스의 시간 계산은 **Asia/Seoul 기준**으로 통일하는 것을 권장한다.

다음 계산 모두 동일한 시간대를 사용한다.

요일 판정
기상목표시간
취침 알림
DND
목표 +15분
30분 쿨다운
사진 8시간
AWAKE / SLEEPING

서버 내부 Timestamp 저장 방식과 별개로 비즈니스 로직은 하나의 명확한 Zone 기준을 가져야 한다.

36. Notification Type

현재 필요한 주요 Notification Type:

BEDTIME_REMINDER
WAKE_REQUEST

BEDTIME_REMINDER

기상목표 기반 취침 유도 알림.

무음
무진동
잠자기 버튼 있음

WAKE_REQUEST

다른 그룹원이 보낸 깨우기 알림.

취침 유도 알림과는 성격이 다르므로 Notification Channel도 분리한다.

37. Notification Status

예약 알림 상태:

PENDING
SENT
FAILED
CANCELLED

PENDING

아직 발송되지 않음.

SENT

FCM 발송 완료.

FAILED

발송 실패.

CANCELLED

더 이상 발송하면 안 됨.

예:

사용자가 잠자기 누름
→ 현재 사이클의 남은 PENDING 알림

→ CANCELLED

또는:

기상목표시간 변경/삭제
→ 기존 목표에 연결된 불필요한 PENDING 알림
→ CANCELLED

38. 공통 에러

HTTP	Code	설명
400	INVALID_WAKE_TARGET_FORMAT	기상목표 입력 형식 오류
400	INVALID_DND_FORMAT	DND 입력 형식 오류
400	INVALID_TIME_RANGE	DND 시간 범위 오류
401	UNAUTHORIZED	인증 필요 / Token 오류
403	WAKE_GROUP_ACCESS_DENIED	그룹 접근 권한 없음
404	WAKE_GROUP_NOT_FOUND	깨우기 그룹 없음
404	WAKE_REQUEST_NOT_FOUND	깨우기 요청 없음
409	ACTIVE_WAKE_GROUP_EXISTS	이미 깨우기 그룹에 가입됨
409	WAKE_GROUP_FULL	그룹 정원 초과
409	ALREADY_MEMBER	이미 해당 그룹 구성원
409	WAKE_BLOCKED_DND	상대방이 DND
409	WAKE_COOLDOWN	상대방 성공 인증 후 30분 이내
409	RETRY_EXHAUSTED	사진 인증 2회 모두 사용
422	POSE_ANALYSIS_FAILED	AI 포즈 판정 실패

기존 공통 에러 구조를 그대로 유지한다. NUNNUN_API_SPEC_v1_REVISIED.pdfPDF

39. 이번 개발에서 제외 / 보류

구분	기능	처리
기존 Auth	Signup / 일반 Login	코드 유지 가능, 데모에서는 미사용
Wake Group	그룹 탈퇴	제외
Multi Group	여러 깨우기 그룹	제외
Multi Group	그룹 간 사진/상태 공유	제외
AI Friend	AI 친구	제외
Payment	결제	추후
Capacity	4 → 8 실제 확장	추후
Roommate	/roommate-* 신규 개발	제외
Reward	리워드	보류

40. 구현 우선순위

P0

```
Demo Auth
↓
Device / FCM
↓
Wake Target
↓
Bedtime Reminder Scheduling
↓
Notification [잠자기]
↓
Sleep
↓
DND
↓
Wake Group 생성 / 참여 / 상세
↓
Wake Request
↓
Daily Pose
↓
Proof
↓
AI 판정
↓
Retry
↓
NEEDS_HELP
↓
Self Verify
```

P1

```
그룹 이름 변경
초대코드 UX
Today 화면
Wake Target 삭제
DND 삭제
알림 스케줄 정리/재생성 안정화
```

P2

```
Fixed Schedule
Sleep Feedback
Stats
계정 부가기능
```


41. 취침 알림 정책 최종 요약

기상목표 = T

T - 9시간

→ 첫 알림

→ "취침 1시간 전이에요."

그 이후

→ 90분마다 알림

알림 내용

→ "기상까지 N시간 N분 남았어요."

T - 1시간 30분

→ 마지막 알림

이후

→ 알림 없음

모든 취침 유도 알림은:

무음

무진동

[잠자기] 버튼

을 사용한다.

잠자기 를 누르면:

POST /me/sleep

↓

잠든 시간 기록

↓

현재 기상목표에 대한 남은 취침 알림 취소

다음 기상목표에서는:

다음 목표 - 9시간

부터 다시 자동으로 알림이 시작된다.

42. Today / Sleep 현재 상태 계약 확장

GET /me/today - sleep 필드 추가

기존 필드는 그대로 유지하고, 현재 활성 수면 상태를 나타내는 sleep 객체를 추가한다.

```
"sleep": {  
  "status": "SLEEPING",  
  "sleep_session_id": 301,  
  "started_at": "2026-08-19T23:40:00+09:00"  
}
```

status: AWAKE | SLEEPING

SLEEPING: 가장 최근 SleepSession 이후 SUCCESS 인증(외부 Wake 또는 Self Verify)이 없음

AWAKE: SleepSession이 없거나, 가장 최근 Session 이후 SUCCESS 인증이 있음

AWAKE일 때 sleep_session_id와 started_at은 null이다.

NEEDS_HELP와 SENT는 성공 인증이 아니므로 SLEEPING 상태를 종료하지 않는다.

started_at 비교와 반환 Offset은 Asia/Seoul 기준이다. 자정이 지나도 같은 활성 Session을 유지한다.

POST /me/sleep - 활성 수면 중복 방지

정상 요청과 201 응답 계약은 기존과 동일하다.

Request: { "source": "APP" } 또는 { "source": "NOTIFICATION" }

Response 201: { "sleep_session_id": 301, "started_at": "...+09:00",
 "bedtime_reminders_cancelled": true }

현재 상태가 SLEEPING이면 새 SleepSession, Roommate 알림, 재취소 작업을 만들지 않는다.

Response 409 Conflict

```
{ "success": false, "error": {  
  "code": "ALREADY_SLEEPING",  
  "message": "User is already sleeping."  
}}
```

동시성과 기상 인증 연결

동일 사용자 POST /me/sleep은 사용자 행 잠금으로 직렬화한다.

외부 Wake Request와 Self Verify는 모두 receiver의 SUCCESS verified_at을 동일하게 사용한다.

현재 Session 이전의 과거 인증은 새 취침 상태에 영향을 주지 않는다.

DB 컬럼, 테이블, Migration은 추가하지 않는다.